

Protección de servicios con *WAF* para *The Store*

Inserción de un WAF en el `ingress-nginx` del cluster local para mitigar los vectores detectados en la auditoría previa, sin modificar el código de los microservicios.

MATERIA

Redes de Información
ITBA · 1C 2026

GRUPO

Grupo 2
Tema 9 · WAF

INTEGRANTES

Mauro Vella · 62134
Enrique Castillo · 68321
Federico Inti García Lauberer · 61374

1 Problemática y contexto

The Store es un sitio de e-commerce compuesto por cinco microservicios —una interfaz web y cuatro backends: catálogo, carritos, pedidos y checkout— que corren sobre un cluster Kubernetes local (Kind) con `ingress-nginx` como único punto de entrada desde Internet. Por diseño los backends no deberían ser accesibles desde afuera; hoy lo son.

Una auditoría del grupo contra `http://localhost` el 16 de abril encontró **10 vectores explotables sin usuario ni contraseña**, en menos de tres minutos con un navegador. Los más graves:

01

ACCESO
INTERNO
EXPUESTO

Los servicios internos se alcanzan desde afuera. El proxy del UI (`/proxy/...`) reenvía requests a los backends, haciendo que carritos, pedidos y checkout —pensados como `ClusterIP`— respondan desde Internet. Un request a `/proxy/carts/admin` devuelve el carrito del usuario admin sin autenticar: IDOR (Insecure Direct Object Reference).

02

PATH
TRAVERSAL

Combinando el proxy con `../` se escapa del backend. Pedir `/proxy/catalog/../../actuador/prometheus` devuelve 25 KB del panel interno: nombres de servicios, capacidad de disco, versión exacta del framework, URLs en vivo y excepciones. Material típico de la fase de reconocimiento de cualquier atacante.

03

SPOOFING
DE IP

Cualquier atacante puede hacerse pasar por cualquier IP. El sitio confía en headers como `X-Forwarded-For` sin verificar el origen. Los logs registran IPs falsas, anula los bloqueos por IP y complica la investigación forense.

04

SIN RATE
LIMIT

No hay límite de velocidad. En una prueba controlada, 100 requests seguidas tardaron 597 ms (167 rps) y todas respondieron `200 OK`. Permite scraping del catálogo, fuerza bruta contra cupones o tokens de sesión, y saturación desde un único origen.

05

ESCANEEO
INVISIBLE

Los escaneos conocidos no se detectan. User-Agents de `sqlmap` y `nikto` pasan sin alarma y devuelven `200 OK`. Un atacante puede mapear todos los puntos débiles sin ser detectado.

06

HEADERS
AUSENTES

Faltan seis headers de seguridad estándar (HSTS, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy). Habilita clickjacking por ausencia de `X-Frame-Options` y permite interceptar sesiones iniciadas sobre WiFi pública por ausencia de HSTS.

Todas las entradas pasan por un único punto (`ingress-nginx`), lo que permite resolver los seis hallazgos en un solo componente sin tocar el código de los cinco microservicios.

2 Diseño de la solución

Se propone desplegar **ModSecurity v3** (motor de inspección HTTP que registra o bloquea requests según reglas) junto con el **OWASP Core Rule Set** (conjunto de ~2000 reglas mantenidas por OWASP que cubren ataques web comunes) dentro del `ingress-nginx-controller` ya presente en el cluster.

Cómo funciona en la práctica: cada request entrante pasa primero por ModSecurity. El motor parsea URL, headers y body, acumula un puntaje de sospecha según cuántas reglas matchean y, si supera el umbral, devuelve `403 Forbidden` en lugar de reenviar al backend. Los cinco microservicios no se enteran del cambio.

ETAPAS DE DESPLIEGUE

01 DetectionOnly (solo registrar): el WAF anota en log qué habría bloqueado. Sirve para descubrir y corregir falsos positivos sobre tráfico legítimo antes de activar el bloqueo real.

02 On (bloquear): una vez tuneado, el WAF pasa a responder `403` sobre las requests maliciosas.

QUÉ AGREGAMOS ENCIMA DEL CRS BASE

- Reglas *custom* para bloquear el panel `/actuator/*` salvo `/actuator/health` (health-check de Kubernetes).
- Reglas *custom* para cerrar el agujero del proxy: denegar cualquier request a `/proxy/*` que contenga `..` en el path o no traiga header de sesión válido.
- Anotaciones en el Ingress para aplicar límite de velocidad por IP y agregar los seis headers de seguridad.

El alcance del cambio queda acotado a archivos de configuración (`ConfigMap` del Ingress + anotaciones). No se modifica el código Java, Go ni Node de los cinco servicios, y cualquier ajuste futuro se hace en un solo lugar.

3 Scope del POC y casos de uso

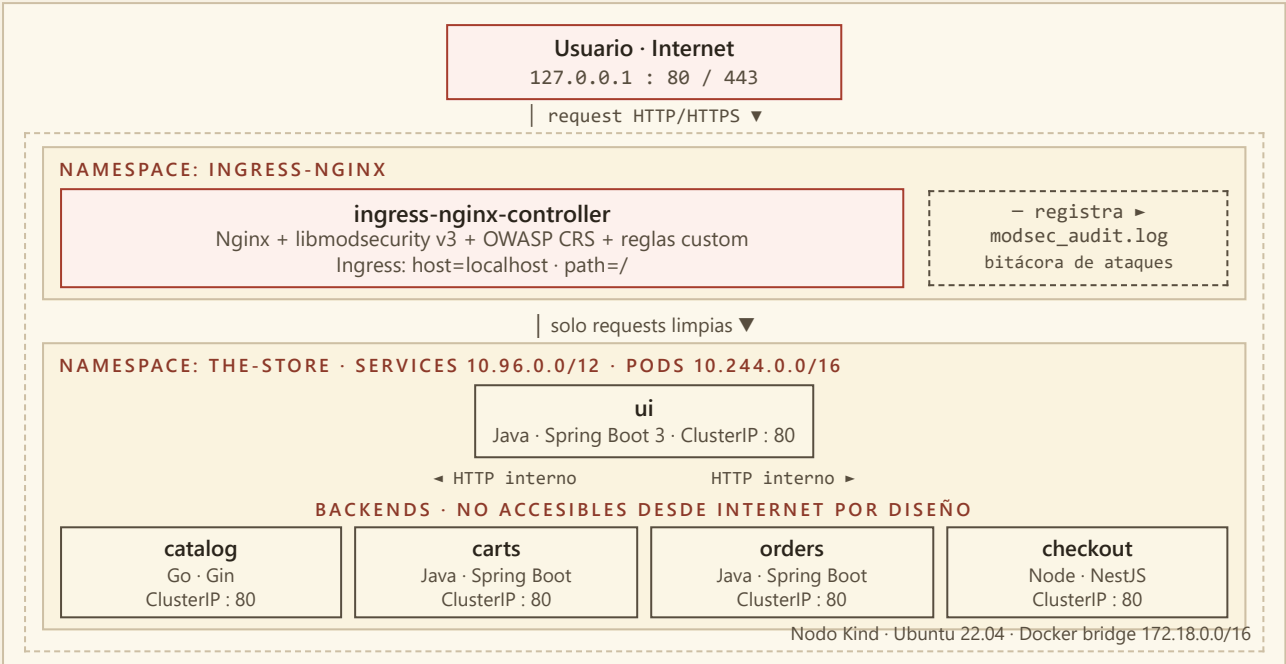
ALCANCE COMPROMETIDO

- Bloqueo de los endpoints de diagnóstico sensibles: `/actuator/info`, `/actuator/metrics` y `/actuator/prometheus` pasan de `200 OK` a `403 Forbidden`.
- Bloqueo de path traversal y SSRF sobre el proxy: payloads como `/proxy/catalog/../../../../actuator/prometheus` quedan cortados por CRS 930xxx y regla custom.
- Detección y bloqueo de User-Agents de herramientas automáticas (`sqlmap`, `nikto`, `nuclei`) mediante CRS 913xxx.
- Límite de velocidad por IP aplicado desde el Ingress sobre el tráfico externo.
- Incorporación de los seis headers de seguridad HTTP en las respuestas servidas a través del Ingress. La `Content-Security-Policy` se entregará en modo `Report-Only` o como política permisiva mientras se valida compatibilidad con la UI.

CASOS DE USO PARA LA DEMO · PATRÓN "ANTES / DESPUÉS"

#	CASO	PRE-WAF	POST-WAF ESPERADO
1	Exfiltración del panel <code>/actuator/prometheus</code>	200 OK · 25 KB	403 Forbidden
2	Path traversal <code>/proxy/catalog/../../../../actuator/info</code>	200 OK	403 Forbidden
3	Request con User-Agent: Nikto/2.5	200 OK	403 Forbidden (logged)
4	Burst de 100 requests en paralelo a <code>/</code>	100 / 100 responden	throttle activo
5	Headers de la home	0 / 6 presentes	6 / 6 (CSP permisiva)

4 Diagrama de arquitectura



PUNTO ÚNICO DE ENTRADA · EL WAF SE INSERTA EN EL INGRESS-NGINX-CONTROLLER

BLOQUE	CIDR	ROL
Host loopback	127.0.0.1/32	Usuario accede a The Store por 80 / 443
Docker bridge del nodo Kind	172.18.0.0/16	Red del contenedor del nodo
Pods del cluster	10.244.0.0/16	Comunicación entre pods
Services ClusterIP	10.96.0.0/12	Direcciones internas de los servicios
SO de los nodos	Ubuntu 22.04 LTS	kindest/node basado en Ubuntu 22.04 LTS
Protocolos	HTTP/1.1	HTTP/1.1 entre usuario/Ingress y entre UI/backends; DNS interno de Kubernetes
Punto de inserción	ingress-nginx	ModSecurity v3 se activa como módulo del Nginx ya presente

5 Alternativas consideradas

Shadow Daemon

DESCARTADO · LISTADO EN EL ENUNCIADO

Requiere conector dentro de cada aplicación y solo provee conectores para PHP, Python y Perl; The Store está escrito en Java, Go y Node.js. Comunidad y base de reglas notoriamente más chicas que OWASP CRS.

OpenWAF

DESCARTADO · LISTADO EN EL ENUNCIADO

Sin commits desde 2018 y documentación parcialmente en chino sin traducir. Herramienta sin mantenimiento activo no es un riesgo asumible para una pieza de seguridad crítica.

WAF gestionado en la nube

DESCARTADO · CLOUDFLARE, AWS WAF, AZURE WAF

La consigna pide implementar un componente propio y desplegarlo en el cluster local; un servicio externo con costo mensual queda fuera de alcance.